

ES Config API — Build Plan

Sep 28, 2026 · @Latish

Overview

A Python/FastAPI service on a private EC2 instance that lets approved users read, change, dry-run and roll back configuration on our self-managed Elasticsearch 8.17.1 clusters, one cluster per request. Every change is snapshotted to S3 first and written to an audit log.

Goals for v1

- Get, update and roll back (one level) every supported config type on any registered cluster
- Dry run on every update and rollback: diff, validation errors, warnings, ES simulation where available
- Per-user, per-cluster view/edit permissions and a settings allowlist, both managed through an admin API
- Record who changed what, when and why

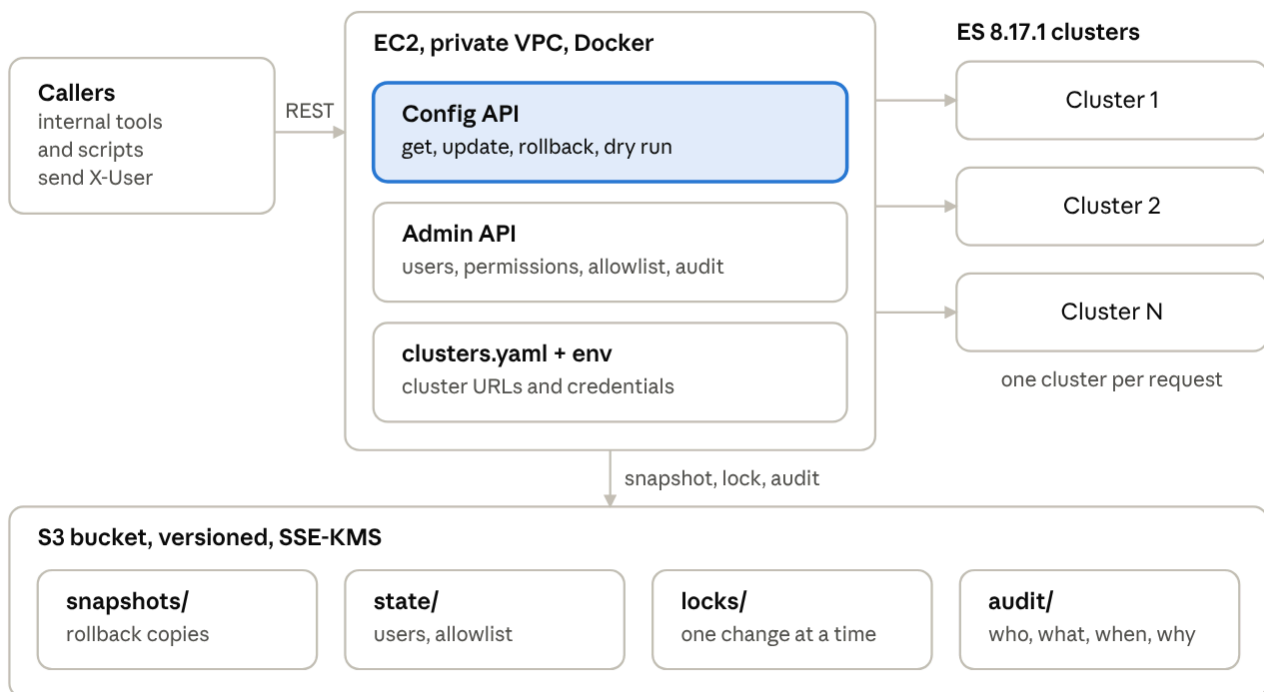
Not in v1

- Real authentication (callers identify with an `X-User` header; swapped for SSO/JWT later)
- Multi-level rollback (the S3 layout keeps history so it can be added without migration)
- Comparing clusters, UI/dashboard, deleting resources, reindex-based mapping rollback

Architecture

One Dockerised FastAPI service on EC2 inside the private VPC; it is the only thing that writes config to the clusters, and S3 is its only data store.

One API on EC2 talks to every cluster and keeps its state in S3



architecture · API on EC2, clusters, S3

- **Clusters** are registered in `clusters.yaml` on the EC2 host; credentials come from environment variables referenced in that file (e.g. `${ES_PROD_PASSWORD}`). Adding a cluster = edit the file + restart the container.
- **S3 holds everything else** as JSON objects. S3 conditional writes (`If-Match` / `If-None-Match`) give us safe concurrent updates and locks, so no database is needed.
- **EC2 reaches S3 through its IAM role** — no AWS keys on the box.

S3 as the data store works well at our scale (a handful of users, changes per day, not per second). The one trade-off: searching the audit log means listing objects by day; if that gets slow we point Athena at the `audit/` prefix, no code change.

Config types and rollback rules

Seven config types, all with get, update and dry run; six support one-level rollback. Mappings on existing indices are add-only and cannot be rolled back.

Config type	Resource	Update style	ES API used	Rollback	Dry-run simulation
Cluster settings	whole cluster	Partial: send only keys to change; <code>null</code> resets a key to its default	<code>_cluster/settings</code> (persistent only)	Yes — restores changed keys, resets added keys to <code>null</code>	Diff + allowlist check
Index settings	one concrete index	Partial, dynamic settings only	<code>{index}/_settings</code>	Yes, same as above	Diff + allowlist check
Index template	template name	Full replace	<code>_index_template/{name}</code>	Yes — old body put back, or deleted if it didn't exist before	<code>_index_template/_simulate</code>
Component template	template name	Full replace	<code>_component_template/{name}</code>	Yes, same as above	Diff
ILM policy	policy name	Full replace	<code>_ilm/policy/{name}</code>	Yes, same as above	Diff
Ingest pipeline	pipeline id	Full replace	<code>_ingest/pipeline/{id}</code>	Yes, same as above	<code>_ingest/pipeline/_simulate</code> with sample docs from the request
Index mapping	one concrete index	Add-only: new fields only	<code>{index}/_mapping</code>	No — dry run and response say "permanent"	Local conflict check

Rules that apply to every type

- Only persistent cluster settings are read or written; transient ones show up as a warning.
- Static index settings (shard count, analyzers, codec and similar) and read-only ones (`index.uuid`, `index.creation_date`, ...) are always rejected, because changing them needs the index closed.
- Index names must be concrete: no wildcards, commas or `_all`.
- Rollback **swaps**: the config being replaced becomes the new snapshot, so a second rollback re-applies the change.

Config API

All paths are under `/api/v1`. Every request carries `X-User: <username>`; the user needs `view` on the cluster to read and `edit` to change, dry-run or roll back.

Method	Path	What it does
GET	<code>/clusters</code>	Clusters this user can see, with their permission level
GET	<code>/clusters/{id}/health</code>	Cluster health (green / yellow / red)
GET	<code>/clusters/{id}/{type}</code>	List resource names (templates, policies, pipelines)
GET	<code>/clusters/{id}/cluster-settings</code>	Current persistent settings + version
PUT	<code>/clusters/{id}/cluster-settings</code>	Update (supports ? <code>dryRun=true</code>)
POST	<code>/clusters/{id}/cluster-settings/rollback</code>	Roll back (supports ? <code>dryRun=true</code>)
GET	<code>/clusters/{id}/cluster-settings/previous</code>	Show the stored snapshot
GET · PUT	<code>/clusters/{id}/indices/{index}/settings</code>	Index settings, same pattern incl. <code>/rollback</code> , <code>/previous</code>
GET · PUT	<code>/clusters/{id}/indices/{index}/mapping</code>	Mapping, add-only, no rollback
GET · PUT	<code>/clusters/{id}/{type}/{name}</code>	<code>index-templates</code> , <code>component-templates</code> , <code>ilm-policies</code> , <code>ingest-pipelines</code> , incl. <code>/rollback</code> , <code>/previous</code>

Update request body (same shape for every type):

```
{
  "config": { "cluster.routing.allocation.enable": "primaries" },
  "reason": "Pause shard moves during node patching",
  "sampleDocs": []
}
```

`config` is the partial settings map for settings types and the full ES body for templates, ILM and pipelines. `reason` is required and goes into the audit log. `sampleDocs` is only used by the ingest-pipeline dry run.

Query options and headers

- `?dryRun=true` — validate and preview, change nothing
- `?force=true` — proceed despite drift or a yellow/red cluster (logged as forced)
- `If-Match: <version>` — optional; the update is refused if the config changed since you read it

Response (update, rollback, dry run) returns `changeId`, `applied`, `versionBefore`, `versionAfter`, a `diff` (added / removed / changed keys with before and after), `warnings`, `simulation` (when available), `rollbackAvailable` and `permanent`.

Status	Meaning
200	Applied, or dry run completed
400	Invalid body or ES rejected the config
403	User lacks permission, or a key is not on the allowlist
404	Unknown cluster or resource, or no snapshot to roll back to
409	Drift detected, cluster unhealthy, or another change in progress (lock)
412	<code>If-Match</code> version doesn't match current config
422	Rollback not supported (mappings) or static setting requested
502	Cluster unreachable

Admin API

A separate set of endpoints under `/api/v1/admin`, usable only by users with `admin: true`. The first admins come from the `BOOTSTRAP_ADMIN` env var so someone can manage users on day one.

Method	Path	What it does
GET	<code>/admin/clusters</code>	All registered clusters (from <code>clusters.yaml</code>) with reachability
GET	<code>/admin/users</code>	All users and their permissions

Method	Path	What it does
GET · PUT · DELETE	<code>/admin/users/{username}</code>	Read, create/replace or remove one user
PUT	<code>/admin/users/{username}/permissions</code>	Set per-cluster access: <code>{"prod-us": "edit", "prod-eu": "view"}</code>
GET · PUT	<code>/admin/allowlist</code>	Global allowlist
GET · PUT · DELETE	<code>/admin/allowlist/{clusterId}</code>	Optional per-cluster override; replaces the global list for that cluster
GET	<code>/admin/audit?date=YYYY-MM-DD&clusterId=&user=</code>	Audit events for one day, filterable

Allowlist format — glob patterns per config type; `deny` wins over `allow`, and anything not allowed is rejected with 403:

```
{
  "cluster-settings": {
    "allow": ["cluster.routing.allocation.*",
"indices.recovery.max_bytes_per_sec"],
    "deny": ["cluster.routing.allocation.enable"]
  },
  "index-settings": { "allow": ["index.number_of_replicas",
"index.refresh_interval"] },
  "index-templates": { "allow": ["logs-*"] },
  "ilm-policies": { "allow": ["*"] }
}
```

For settings types the patterns match setting keys; for templates, policies and pipelines they match resource names; for mappings they match index names. A type missing from the allowlist is fully locked.

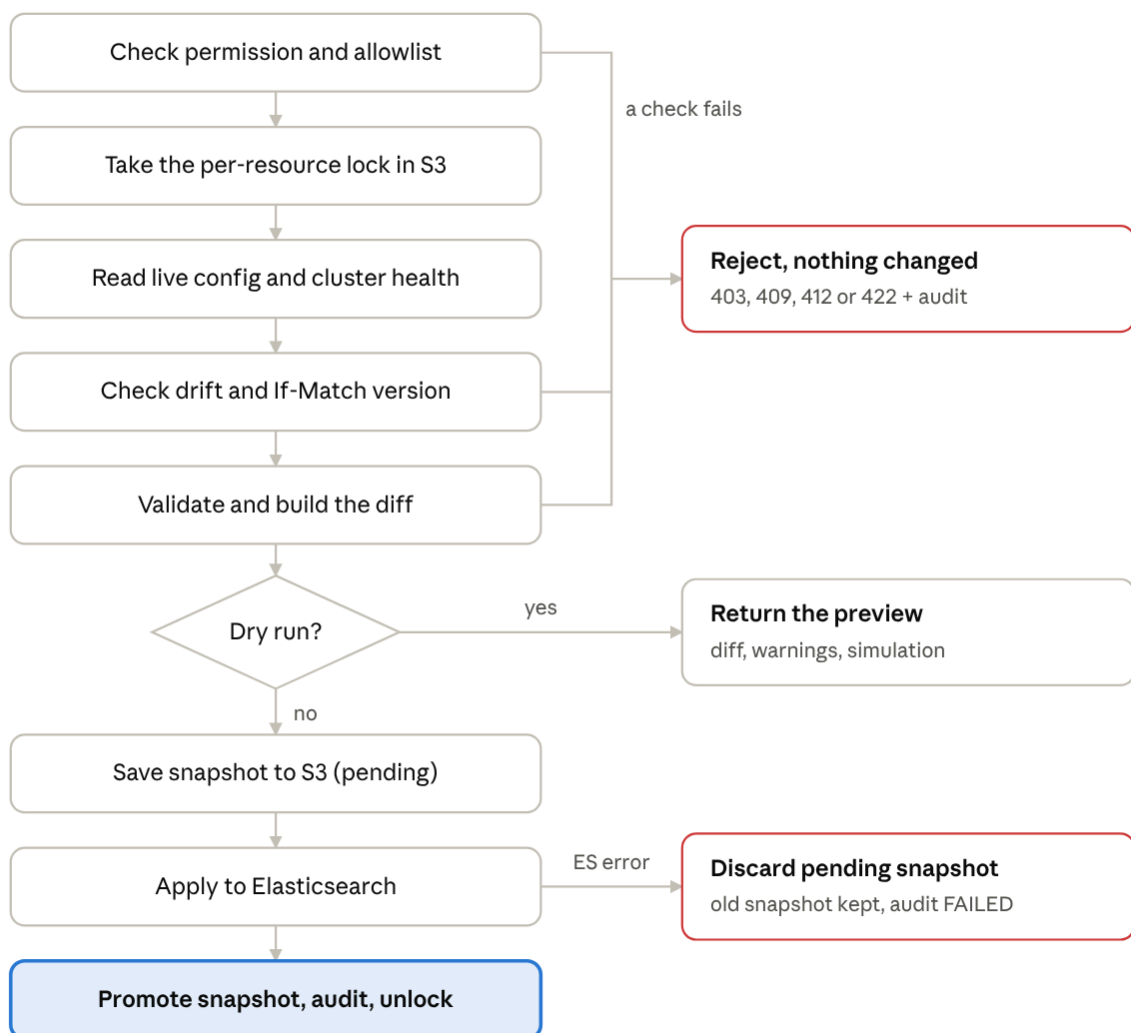
Admin changes (users, permissions, allowlist) are written to the audit log too.

Update and rollback flows

Nothing is written to Elasticsearch until every check passes and the snapshot is saved; a failure at any point leaves the previous snapshot intact.

An update touches Elasticsearch only after every check passes

and the snapshot is safely in S3



update flow · 9 steps, 3 exits

The snapshot is written as **pending** first and promoted only after ES accepts the change. If the service crashes in between, the next request compares the live config with the pending one and either promotes it or discards it, so the rollback copy is never lost.

Rollback follows the same path with two differences: the target config is the stored snapshot, and drift blocks it (unless `force=true`) because rolling back would silently wipe a change made outside the API. After success the two configs swap: what was live becomes the new snapshot.

S3 layout (bucket from `S3_BUCKET`, all keys under `S3_PREFIX`):

Key	Holds
<code>snapshots/{cluster}/{type}/{resource}.json</code>	previous config, pending change, lastApplied hash, who/when
<code>locks/{cluster}/{type}/{resource}.lock</code>	Lock owner and expiry (default 120 s)
<code>state/users.json</code>	Users, admin flag, per-cluster permissions
<code>state/allowlist/global.json</code> , <code>state/allowlist/{cluster}.json</code>	Allowlists
<code>audit/{yyyy}/{mm}/{dd}/{timestamp}_{eventId}.json</code>	One object per audit event

Cluster settings use `_cluster` as the resource name. Bucket versioning keeps every older snapshot, so multi-level rollback later needs no data migration.

Safety

Five guards, all enforced by the API before it writes anything:

Guard	How it works	When it trips
Permissions	<code>X-User</code> must exist in <code>state/users.json</code> with <code>view</code> or <code>edit</code> on the cluster	403
Allowlist	Every changed key (settings) or resource name (templates, ILM, pipelines, mappings) must match <code>allow</code> and not <code>deny</code>	403, listing the blocked keys
Lock	Per-resource lock object in S3, created with <code>If-None-Match: *</code> ; expires after 120 s so a crash can't block forever	409 <code>CHANGE_IN_PROGRESS</code>
Drift detection	Hash of live config compared with the hash the API last wrote	Update: warning, proceeds (the snapshot captures the real live state). Rollback: 409 <code>DRIFT_DETECTED</code> unless <code>force=true</code>

Guard	How it works	When it trips
Cluster health	<code>_cluster/health</code> checked before applying	Red: 409 unless <code>force=true</code> . Yellow: warning

Locking down direct writes (the other half of "do both"). This is set up once on each cluster, outside the API:

- Create an ES role `config_api_writer` with `manage`, `manage_ilm`, `manage_index_templates`, `manage_pipeline` and index `manage` privileges; only the API's service account gets it.
- Give people read-only roles (`monitor`, `view_index_metadata`, `read`), so Kibana Dev Tools can inspect but not change config.
- The repo ships these role definitions as ready-to-run requests in `docs/es-lockdown.md`.

Header identity is not authentication. Anyone who can reach the API can claim any username, so until SSO/JWT is added the EC2 security group must allow only trusted internal sources. Identity lives in one function, so swapping it later touches one file.

Audit logging

Every update, rollback, dry run, rejection and admin change writes one JSON object to `audit/{yyyy}/{mm}/{dd}/` in S3. The schema sits in one module, so it can follow the logging design you're finalising.

Field	Example
<code>eventId</code> , <code>timestamp</code> (UTC ISO-8601)	<code>9f1c...</code> , <code>2026-10-05T09:14:02Z</code>
<code>actor</code> , <code>sourceIp</code> , <code>requestId</code>	<code>latish</code> , <code>10.0.4.21</code> , <code>req-7a1...</code>
<code>action</code>	<code>UPDATE</code> , <code>ROLLBACK</code> , <code>DRY_RUN</code> , <code>ADMIN_USER_UPDATE</code> , <code>ADMIN_ALLOWLIST_UPDATE</code>
<code>clusterId</code> , <code>configType</code> , <code>resource</code>	<code>prod-us</code> , <code>cluster-settings</code> , <code>_cluster</code>
<code>outcome</code> , <code>error</code>	<code>SUCCESS</code> / <code>FAILED</code> / <code>REJECTED</code> , error code + message
<code>reason</code>	Free text from the request
<code>versionBefore</code> , <code>versionAfter</code>	Config hashes

Field	Example
<code>diff</code>	Added / removed / changed keys with before and after values
<code>driftDetected</code> , <code>forced</code> , <code>snapshotKey</code>	<code>false</code> , <code>false</code> , S3 key of the snapshot

The same event also goes to stdout as a JSON line, so CloudWatch Logs picks it up if the agent is installed. For tamper-proof logs, turn on S3 Object Lock for the `audit/` prefix (governance mode, retention you choose).

Tech stack and project structure

Python 3.12, FastAPI, official `elasticsearch` 8.17 client, `boto3` for S3, Docker. Interactive API docs are served at `/docs`.

```

es-config-api/
  app/
    main.py          FastAPI app, error handlers
    settings.py      env configuration
    clusters.py      clusters.yaml loader (${ENV} substitution) + ES
  clients
    identity.py      X-User -> user, permission checks (swap for SSO
later)
    storage.py       S3 object store with conditional writes (+ local
store for dev)
    repos.py         users, allowlist, snapshots, locks, audit
    handlers/        one module per config type: fetch, validate, diff,
apply, rollback plan, simulate
    service.py       update / rollback / dry-run orchestration
    routes_config.py config API
    routes_admin.py  admin API
    util.py          diff, hashing, glob matching
  config/clusters.example.yaml
  tests/             run against a real ES 8.17.1 + S3-compatible store
  docs/es-lockdown.md ES roles for the lockdown
  docs/iam-policy.json EC2 role policy for the bucket
  Dockerfile, docker-compose.yml, .env.example, requirements.txt, README.md

```

Environment variables

Variable	Purpose	Default
<code>S3_BUCKET</code> / <code>S3_PREFIX</code> / <code>AWS_REGION</code>	Where state lives	<code>— / es-config-api/ / —</code>
<code>CLUSTERS_FILE</code>	Path to clusters.yaml	<code>/app/config/clusters.yaml</code>
<code>BOOTSTRAP_ADMINIS</code>	Comma-separated admin usernames	<code>—</code>
<code>USER_HEADER</code>	Header carrying the username	<code>X-User</code>
<code>LOCK_TTL_SECONDS</code>	Lock expiry	<code>120</code>
<code>ES_*</code>	Per-cluster credentials referenced from clusters.yaml	<code>—</code>

Build phases

Phases 1–4 are being built now; phase 5 needs your environment.

1. **Foundation** — project skeleton, settings, clusters.yaml loader, S3 store with conditional writes, ES client per cluster.
2. **Config types** — handlers for all seven types: fetch, normalise, validate, diff, apply, rollback plan, simulation.
3. **Config API + safety** — list clusters, get / update / rollback / dry run, locks, drift detection, health gate, allowlist, audit events.
4. **Admin API + packaging** — users, permissions, allowlist, audit query; Dockerfile, compose, README, ES lockdown roles, IAM policy; tests against a real ES 8.17.1.
5. **Rollout (your side)** — create the S3 bucket, attach the IAM role to EC2, fill clusters.yaml + env, apply the ES lockdown roles, seed admins and the allowlist, run the smoke test on a non-production cluster first.

Open items

- ☐ Confirm rollback **swaps** (current becomes the new snapshot) rather than consuming the snapshot
- ☐ S3 bucket name and AWS region
- ☐ Cluster list: id, display name, URL, auth type (basic or API key), CA certificate
- ☐ Initial allowlist contents per config type

- ☐ First admin usernames for `BOOTSTRAP_ADMINS`
- ☐ Final logging design — map it onto the audit event schema above
- ☐ Audit retention period and whether to turn on S3 Object Lock
- ☐ Timeline for real authentication (SSO / JWT) to replace the `X-User` header